

App de gastos

Implementación en Kotlin & Android

Implementación del proyecto



Modelado de Datos: Clases y Enums



Persistencia con SharedPreferences



Generación de Reportes PDF



UI con Jetpack Compose



Visualización de Datos (Canvas)



Lógica de Negocio y KPIs



Filtrado y Ordenación



Conclusiones y Buenas Prácticas

1. Estructura y Modelos

Definiendo el ADN de nuestra aplicación

Data Classes: El poder de Kotlin

La clase **Gasto** utiliza una *Data Class* para encapsular la información financiera. Es la unidad mínima de nuestra app.

Inmutabilidad: Los datos se definen con 'val'.

Utilidades: Autogeneración de toString(), equals() y hashCode().

```
data class Gasto(  
    val id: Long,  
    val concepto: String,  
    val cantidad: Float,  
    val categoria: String,  
    val fecha: String  
)
```

Uso de Enum para categorización



Control

Evita errores tipográficos al registrar gastos. Categorías cerradas.



Estética

Asocia cada categoría a un color de Compose (Color) directamente.



Filtrado

Facilita la agrupación de datos para los motores gráficos.

2. Almacenamiento de Datos

Haciendo que la información sobreviva al cierre de la app

SharedPreferences como Base de Datos

Lite

Ideal para Aprendizaje

En lugar de una compleja base de datos SQL (Room), usamos un sistema de Clave-Valor más sencillo.

Serialización manual: Convertimos objetos en texto usando separadores (| y ;).

Acceso asíncrono: Implementado mediante `.apply()`.

El protocolo de datos "Pipe"

Objeto Gasto	Representación en String (DB)
Compra Comida (45.50€)	1711284000 Compra Comida 45.50 Alimentación 24/03/2026
Separador de Bloques	Utilizamos el punto y coma (;) para distinguir cada registro.

3. Exportación e informes

Saliendo del entorno digital al mundo físico

Generación de PDF Nativo

PdfDocument

Usamos la API nativa de Android para "dibujar" en un lienzo virtual de tamaño A4 (595x842 puntos).

Sistema de Archivos

Guardamos en `DIRECTORY_DOWNLOADS` para que el usuario acceda al archivo sin permisos especiales.

Auto-Apertura y StrictMode

- ⚠ **El Reto:** Android bloquea el acceso directo a archivos por seguridad (`FileNotFoundException`).
- 🔗 **La Solución Educativa:** Usamos `StrictMode.setVmPolicy` para relajar las reglas en esta app de aprendizaje.
- 🔗 **Intent Chooser:** Invocamos al sistema para que el usuario elija su lector de PDF favorito.

4. Interfaz con Jetpack Compose

Declaratividad y reactividad en estado puro

| Single Page Application (SPA)

En lugar de múltiples *Activities*, usamos un único **MainActivity** que gestiona el estado de navegación.

Estado Mutable: *pantallaActual* define qué Composable se renderiza.

BackHandler: Gestionamos el botón físico "atrás" de Android para no cerrar la app accidentalmente.

Dashboard	Analítica y KPIs
Nuevo Gasto	Formulario de entrada
Historial	Lista filtrable

Diseño del Dashboard



Resumen

Visualización del gasto total frente al presupuesto mensual mediante Cards.



Tendencias

Integración de Canvas para gráficos de barras y circulares.



Acciones

Botones flotantes para añadir gastos rápidamente.

5. Lógica de Negocio Financiera

Transformando datos brutos en información inteligente

Cálculo de KPIs Analíticos

Avg

Promedio Diario Inteligente

Calculado dividiendo el total gastado entre el número de días únicos presentes en el historial.

```
total / gastos.map { it.fecha }.distinct().size
```


Predicción de Flujo de Caja

Estimación: Multiplicamos el promedio diario por 30 para prever el gasto a fin de mes.

 **Regla del 80%:** Si el gasto actual supera el 80% del presupuesto, la app cambia a un color de alerta (Naranja/Rojo).

 **Categoría Top:** Uso de *groupBy* y *maxByOrNull* para identificar el mayor foco de gasto.

6. Visualización con Canvas

Geometría aplicada a las finanzas personales

El Gráfico Circular

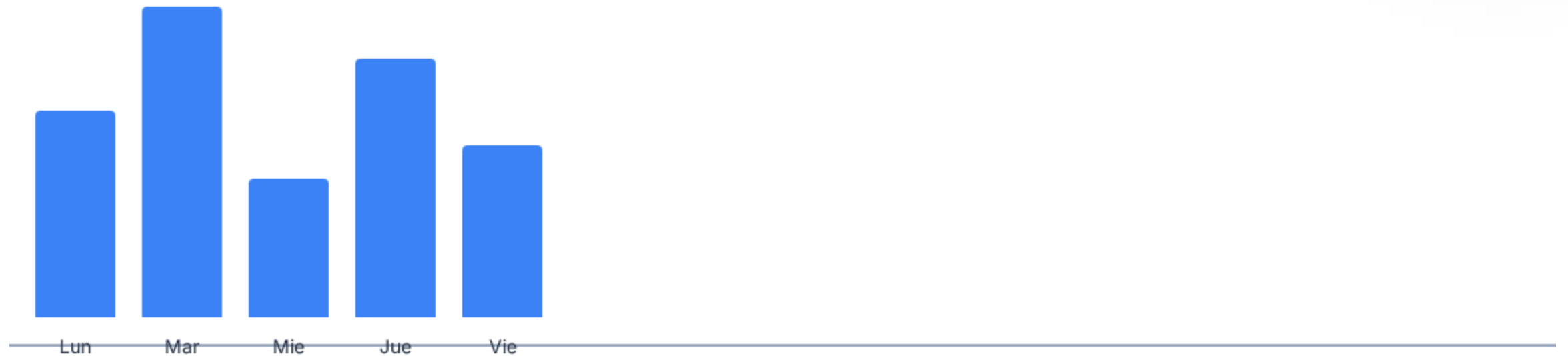
drawArc: Usamos ángulos para representar porcentajes.

Lógica: 360 grados equivalen al 100% del gasto total.

Leyenda Dinámica: Sincronización de colores entre el gráfico y el texto.



Gráfico de Barras Verticales



Visualización de los últimos 5 días de gasto.

Native Canvas vs Compose Canvas

El Reto del Texto

El Canvas moderno de Compose es excelente para figuras, pero limitado para texto avanzado. Usamos **nativeCanvas** para acceder a las APIs clásicas de Android (Paint) y rotular los ejes de los gráficos.

7. Interacción con el Usuario

Componentes avanzados

Integración de DatePickerDialog

Evita que el usuario escriba fechas mal formateadas.

Utiliza el calendario nativo del sistema operativo.

Sincronización: Se actualiza el *state* 'fechaSeleccionada' al cerrar el diálogo.



24/03/2026

Filtrado y ordenación dinámica

El historial no es una lista estática. Aplicamos transformaciones funcionales de Kotlin en tiempo real:

Filtrado

```
.filter { it.categoria == seleccion }
```

Ordenación

```
.sortedByDescending { it.cantidad }
```


Validación de entrada de datos

-  **KeyboardType.Decimal:** Forzamos al sistema a mostrar el teclado numérico.
-  **Sanitización:** Comprobamos que el concepto no esté vacío y el monto sea > 0 antes de guardar.
-  **Feedback:** Uso de *Toast* para confirmar al usuario que el registro fue exitoso.

8. Conclusiones finales

Aprendizajes clave del desarrollo de MyBudget

Arquitectura y Estado

Unidireccional

El estado fluye hacia abajo y los eventos hacia arriba (Hoisting).

Modularidad

Separación clara entre UI (MainActivity), Datos (Gestor) y Lógica (KPIs).

Reactividad

La interfaz se actualiza sola al modificar las listas de *remember*.

| Consejos

"No intentes construir Room o Retrofit el primer día. Domina primero el estado de Compose y la manipulación de colecciones en Kotlin."

Próximos Pasos

Fase 1	Sustituir SharedPreferences por Room (SQLite)
Fase 2	Implementar Biometría (Huella) para seguridad
Fase 3	Sincronización en la nube con Firebase